The Arithmetic and Logic Unit

Lecture 3

ALU – Arithmetic and Logic Unit

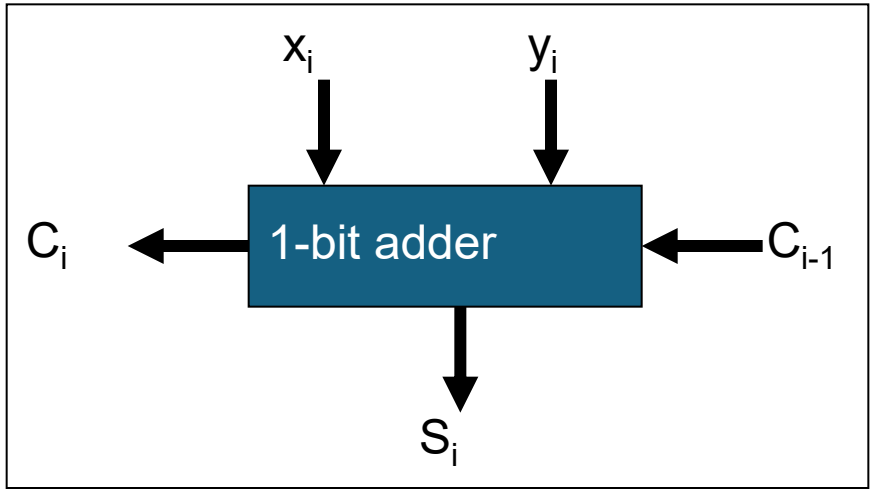
- Purpose: computes arithmetic and logic operations:
  - arithmetic:
    - basic operations: addition, subtraction, multiplication, division, modulo
    - special functions: exponential, logarithm, sine, cosine, tangent, arctangent, etc.
  - logic:
    - AND, OR, NOT, XOR
- Types of arithmetic units:
  - integer arithmetic
  - floating-point arithmetic (e.g., FPU)
  - signal-processing arithmetic (e.g., MMX with saturation)
  - parallel arithmetic (MMX – integer, SSE2 – floating point)

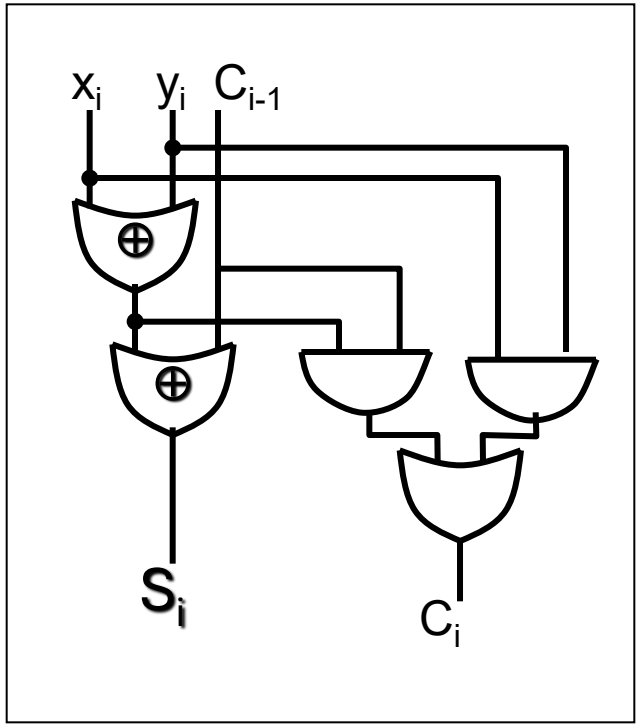
Addition

- the most frequently used operation
- all other arithmetic operations are based on addition:
  - subtraction – addition of the two's complement
  - multiplication – repeated addition
  - division – repeated subtraction and addition
- an efficient implementation of addition:
  - directly influences all other operations
  - efficiency: speed and cost (complexity)

Addition

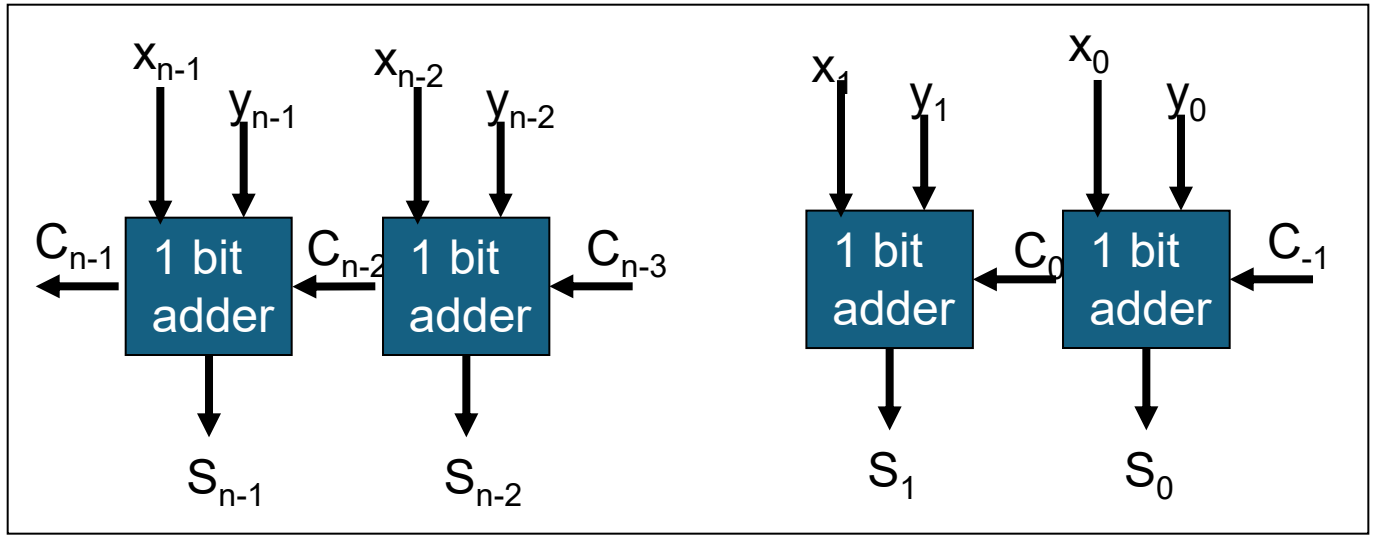
- Basic (full) adder unit – 1-bit addition
  - inputs: x_i, y_i, C_{i-1}
  - outputs:
    - $S_i = x_i \oplus y_i \oplus C_i$
    - $C_i = x_i y_i + (x_i \oplus y_i) C_{i-1}$
  - delay: 3 * gate_delay

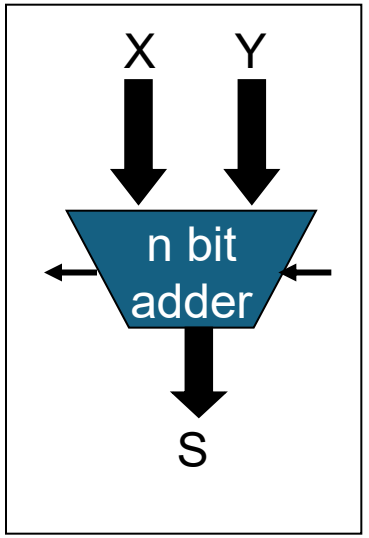




n-bit Ripple-Carry Adder

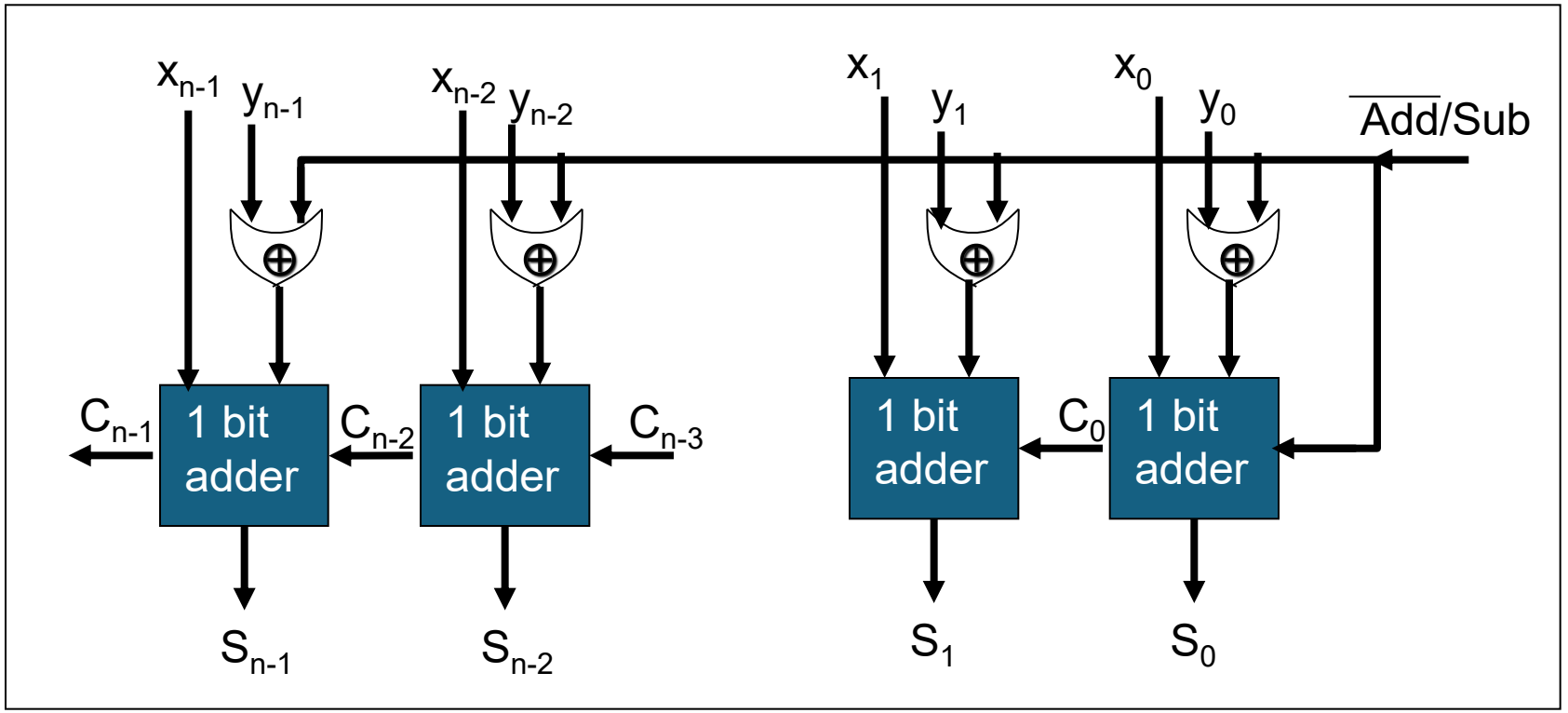
- n-bit adder = n × (1-bit adder)
- Delay: n * 3 * gate_delay
  - example:
    - n = 32; gate_delay = 10 ns (TTL gate) =>
    - delay: 32*3*10ns ~= 1000 ns => f_{clk_max} = 1/1000 ns = 10^6 =1MHz !!!



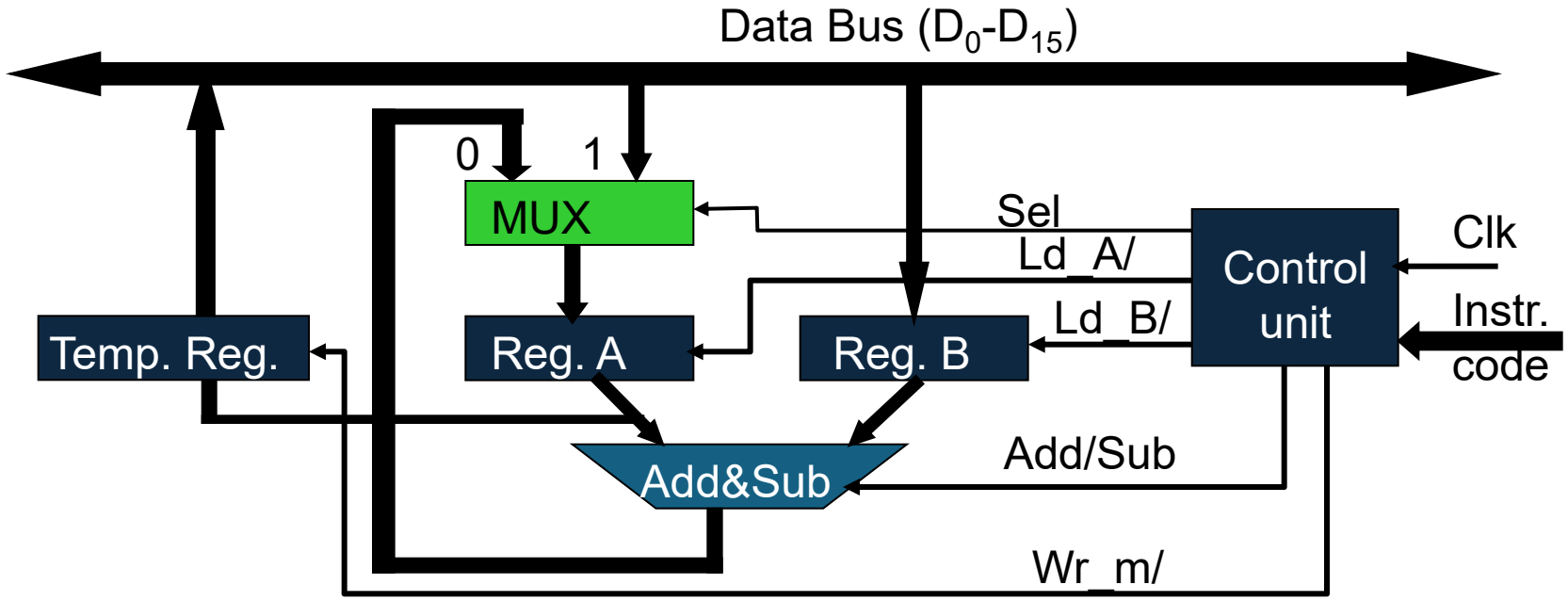


Subtraction

- Subtraction = addition of the two's complement of the second operand
- n-bit adder/subtractor:
  - Add/Sub = 0 => addition
  - Add/Sub = 1 => subtraction



Sequence of Steps for Addition



Step	*BUS*	*SEL*	*LD_A/*	*LD_B/*	*Add/Sub*	*Wr_m/*	*Result*
1	X	1	0	1	-	1	A<=X
2	Y	0	1	0	0	1	B<=Y
3	-	0	0	1	0	1	A<=X+Y
4	Z	-	1	1	-	0	Z<=X+Y

Carry Lookahead Adder (CLA)

- Problem: the carry propagation delay
- Solution: generate the carry directly

$$C_i = x_i y_i + (x_i \oplus y_i)\, C_{i-1} = g_i + p_i * C_{i-1}$$

where: g_i – carry generate

p_i – carry propagate

$C_0 = x_0 y_0 + (x_0 \oplus y_0) C_{-1} = g_0 + p_0 * C_{-1}$

$C_1 = x_1 y_1 + (x_1 \oplus y_1) C_0 = g_1 + p_1 * C_0 = g_1 + p_1 * (g_0 + p_0 * C_{-1}) = g_1 + p_1 g_0 + p_1 p_0 C_{-1}$

$C_2 = x_2 y_2 + (x_2 \oplus y_2) C_1 = g_2 + p_2 * C_1 = g_2 + p_2 * [g_1 + p_1 * (g_0 + p_0 * C_{-1})] =$

$= g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 * C_{-1}$

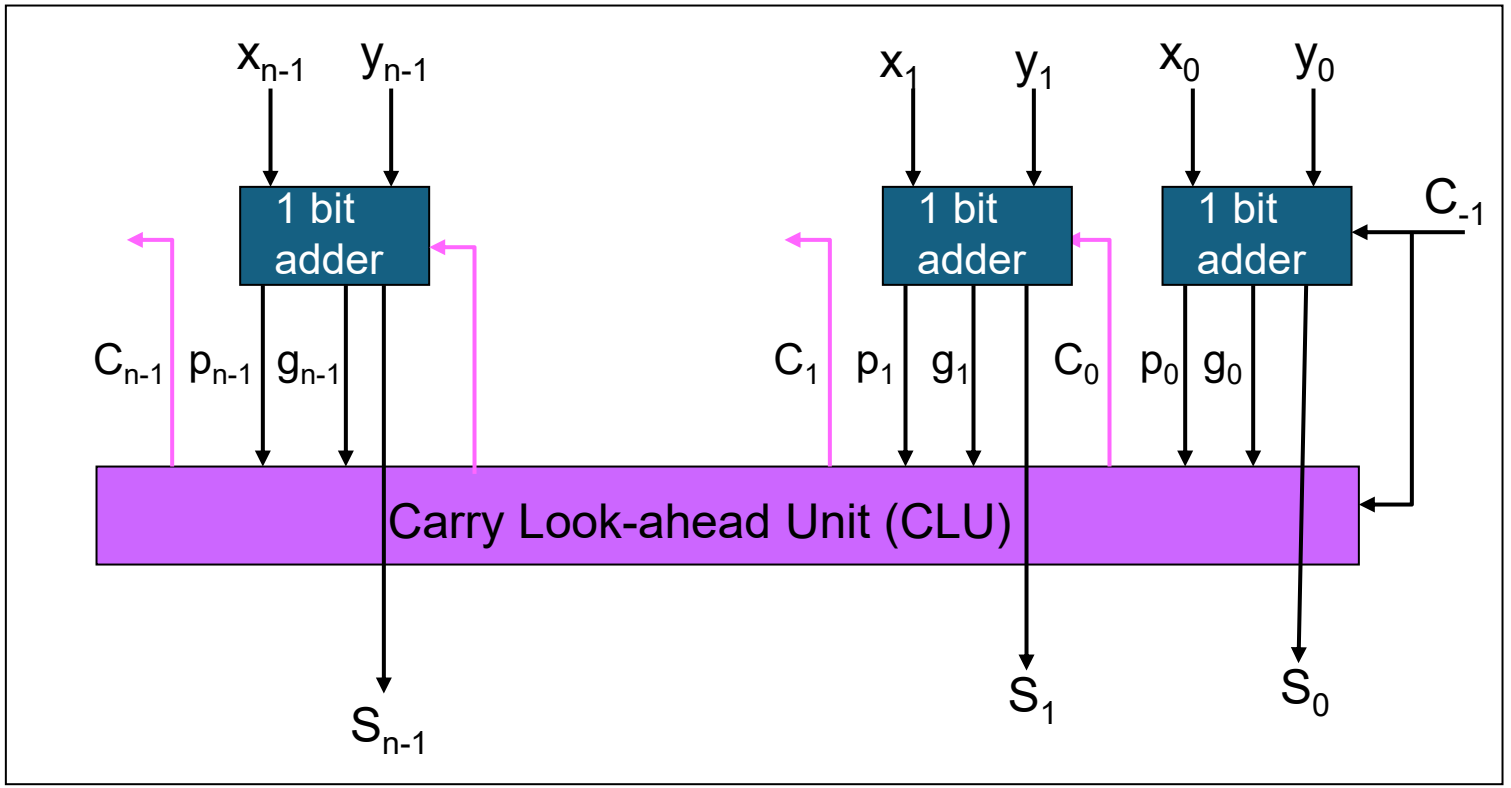
.....

$C_i = f(g_0, g_1, \ldots g_i, p_0, p_1, \ldots p_i, C_{-1}) = f(x_0, x_1, \ldots x_i, y_0, y_1, \ldots y_i, C_{-1})$

- Conclusion: C_i is obtained directly, ONLY by combining the input signals

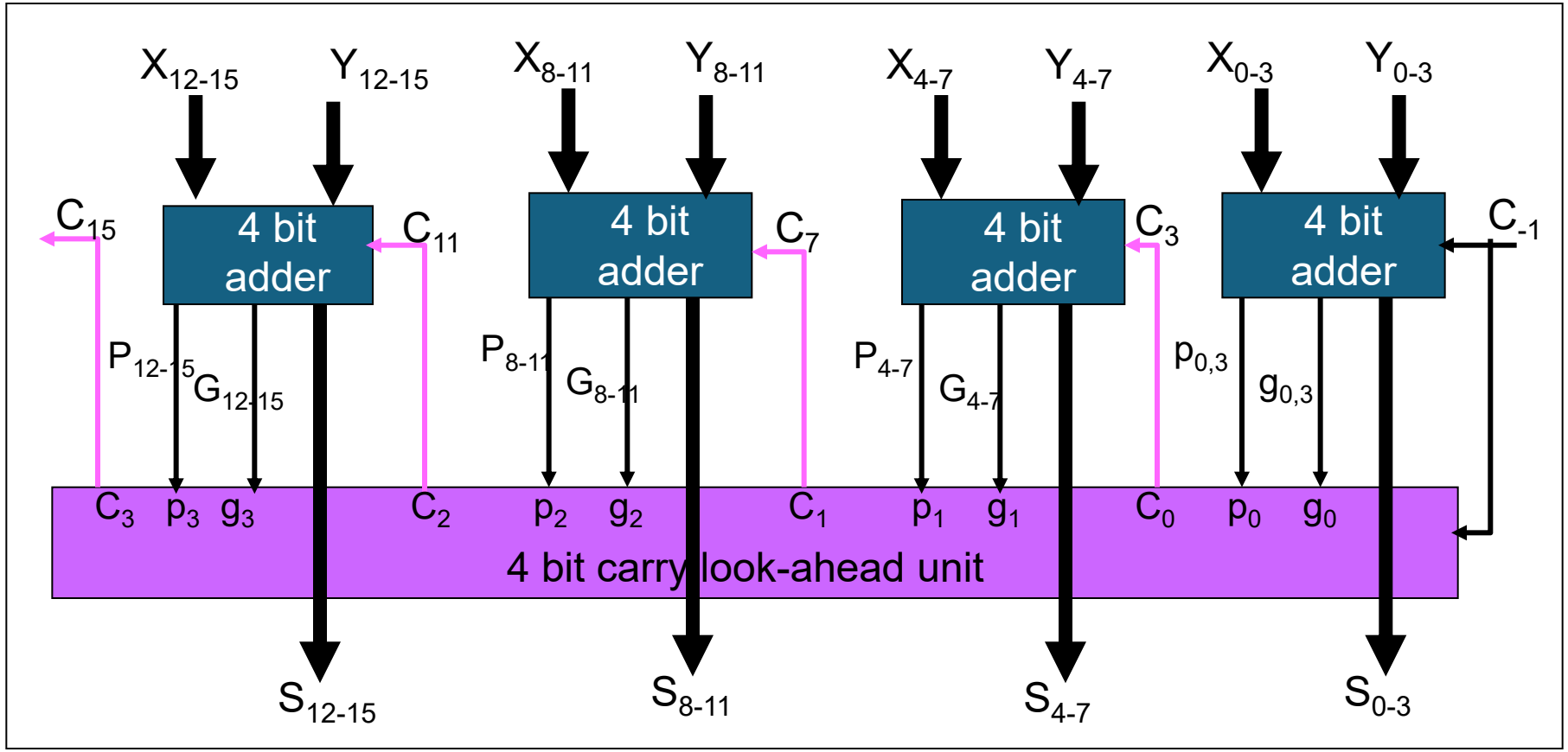
- Disadvantages:
  - circuit complexity grows exponentially with the number of bits (n)
  - gates with a large number of inputs are needed
  - ideal delay = 2 * gate_delay

Carry Look-ahead Unit- CLU



- produces the result in a shorter time
- the CLU is feasible for 4 bits – the number of inputs per gate is limited
- it can be extended by cascading 4-bit adders

Carry Look-ahead Adder



- 16-bit carry lookahead adder built from:
  - 4 × 4-bit carry lookahead adders
  - one 4-bit carry lookahead unit

Carry Look-ahead Adder

- extension from 4 bits to 16 bits
- Generate and propagate signals for bit blocks from "i" to "k":
  - Group generate $g_{i,k}$
  - Group propagate $p_{i,k}$
- For a 4-bit block:

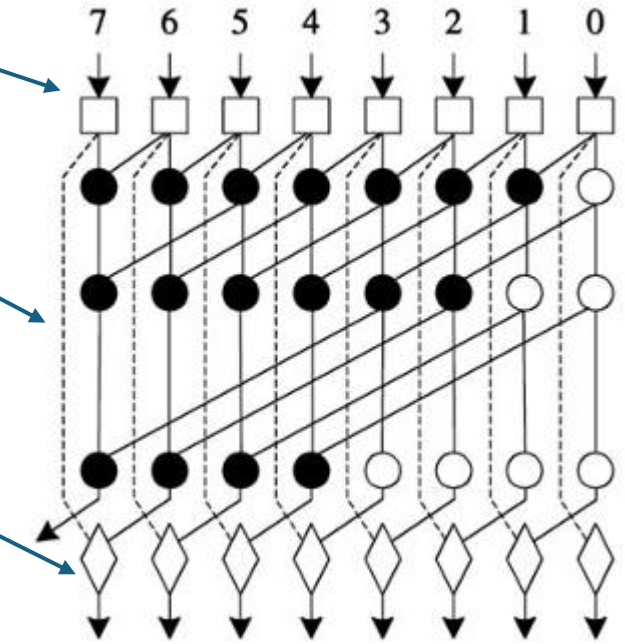
  $G_{3,0} = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0$

  $P_{3,0} = p_3 p_2 p_1 p_0$
- Using this notation we obtain the block carries C_3, C_7, C_{11}, C_{15}

  $C_3 = G_{3,0} + P_{3,0} C_{-1}$ $\quad C_7 = G_{7,4} + P_{7,4} C_3 = G_{7,4} + P_{7,4}(G_{3,0} + P_{3,0} C_{-1})$

Parallel prefix adders (Kogge-Stone)

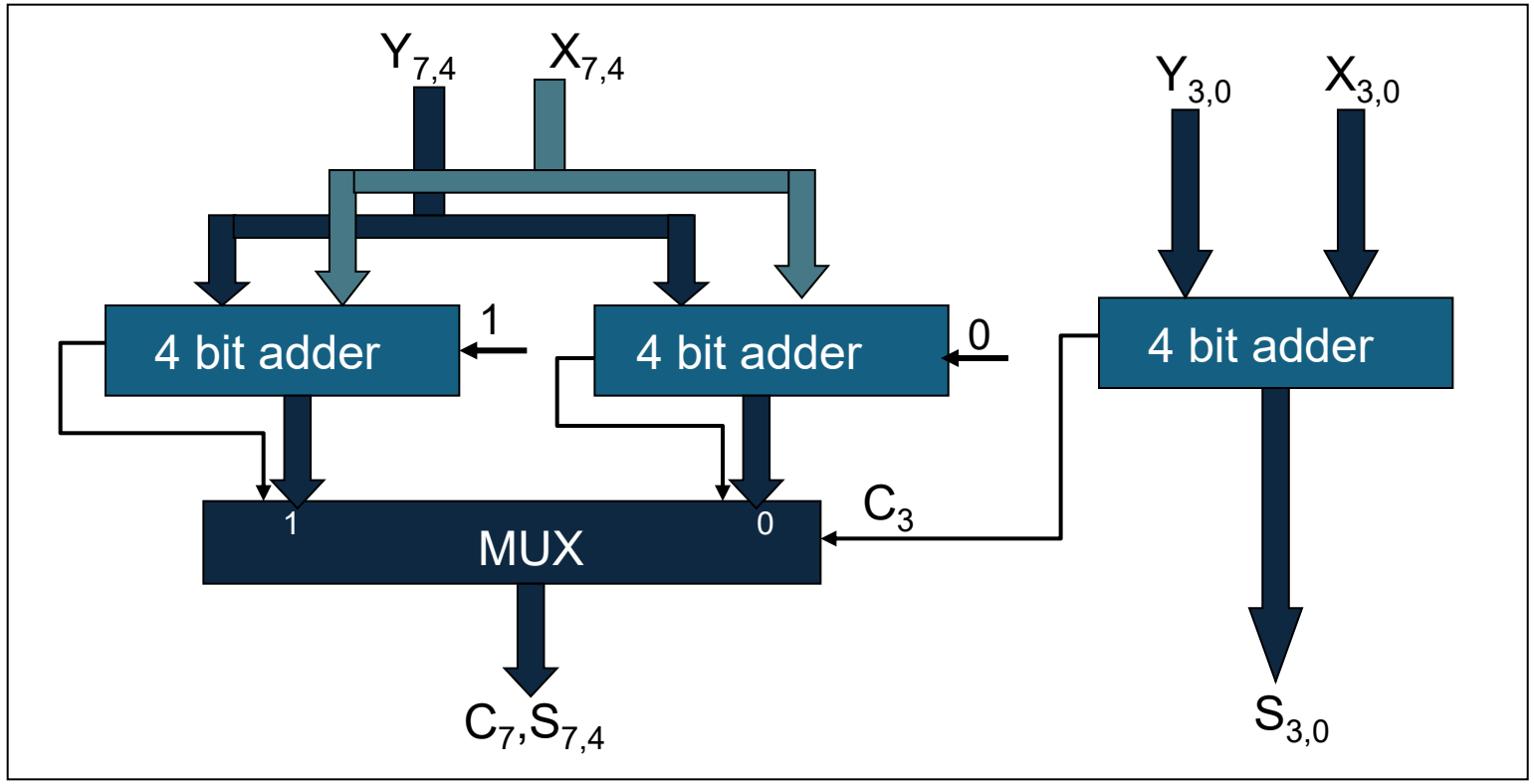
- Precomputation: $g_i = x_i y_i$; $p_i = x_i \oplus y_i$
- Prefix: $g_{i,j} = g_{i,k} + p_{i,k}\, g_k$; $p_{i,j} = p_{i,k}\, p_k$
- Postcomputation: $s_i = p_i \oplus g_{i,0}$



Dimitrakopoulos, Giorgos, and Dimitris Nikolos. "High-speed parallel-prefix VLSI Ling adders." IEEE Transactions on Computers 54.2 (2005): 225-231.

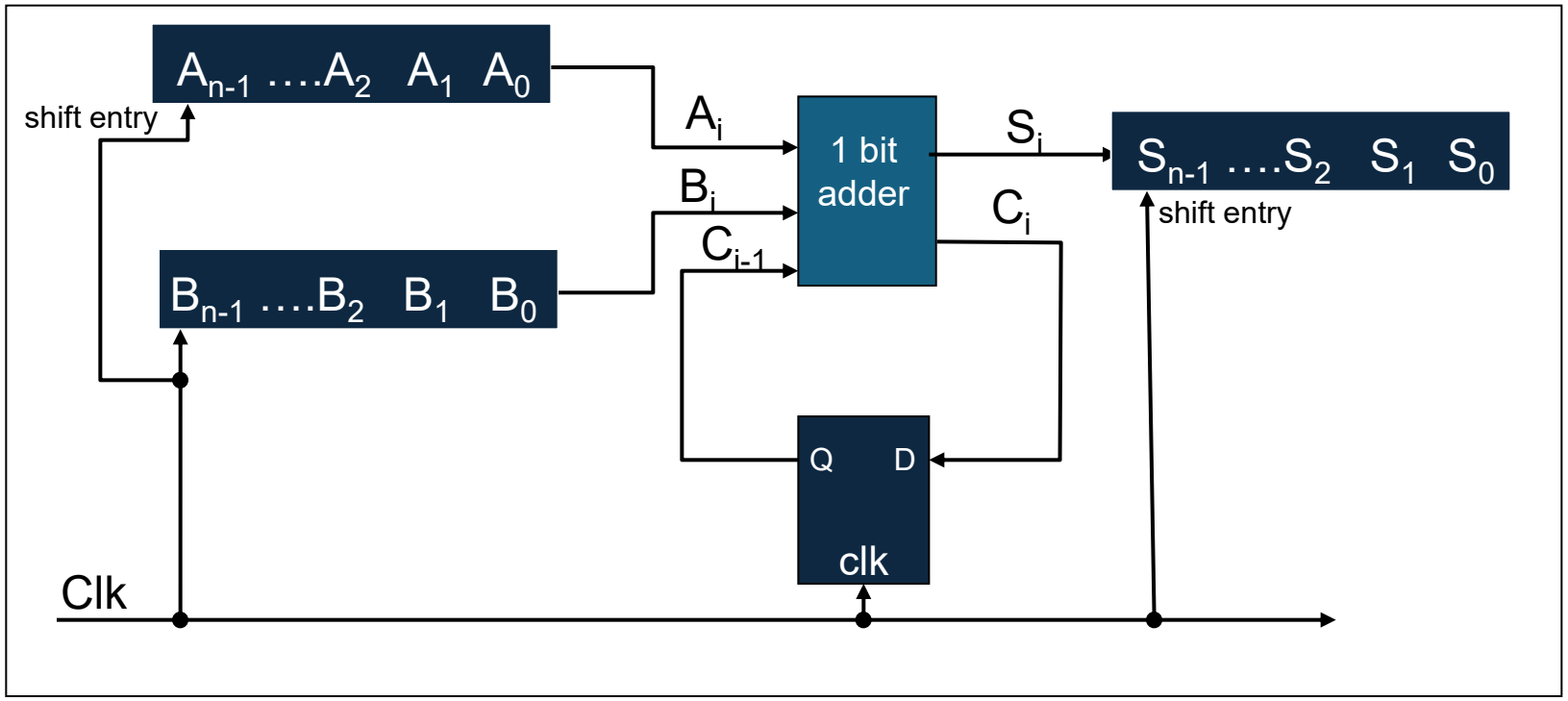
Carry-Select Addition

- Additional hardware to speed up addition
- Avoids using a complex CLA circuit



Serial Adder

- Adds two bit sequences using a single 1-bit adder

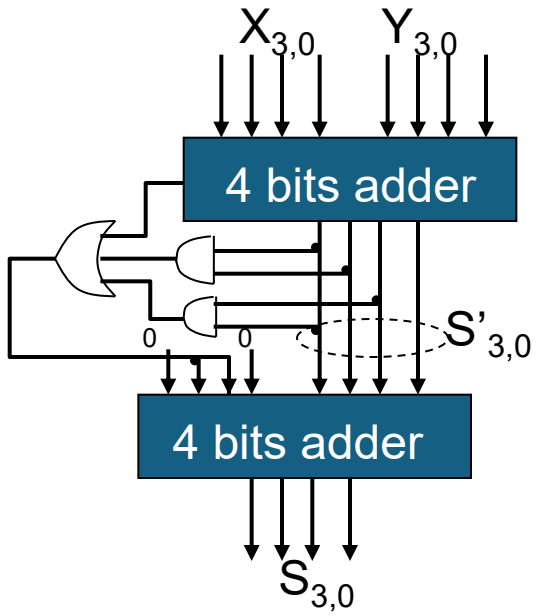


BCD Adder

- addition of numbers in BCD (binary-coded decimal) representation
  - a correction is needed:
    - if the digit is not a valid decimal digit
    - if a carry is generated into the next 4-bit group (the next decimal digit)
  - solution: add 6 (in both cases)
  - Example:

```
  89+
  42
 ----
  CB+  correction
  66
 ----
 131
```

S_3	S_2	S_1	S_0	C	Corr
0	0	0	0	0	0
1	0	0	1	0	0
1	0	1	0	0	1
1	0	1	1	0	1
1	1	0	0	0	1
1	1	0	1	0	1
1	1	1	0	0	1
1	1	1	1	0	1
x	x	x	x	1	1



Saturating Addition and Subtraction

- Idea: if an addition or subtraction overflows or underflows, the result should be the maximum or minimum representable value
- example:
  - 8-bit unsigned representation

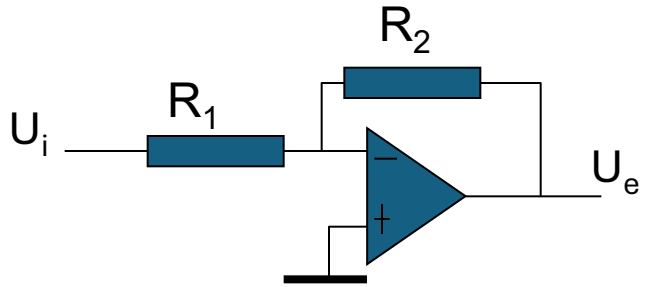
    Normal addition (wraparound)
    80h+90h = 10h (error, overflow)
    80h-90h = F0h (underflow)

    With saturation
    80h+90h = FFh (maximum value)
    80h-90h = 00h (minimum value)
  - 8-bit signed representation

    Normal addition (wraparound)
    70h+20h = 90h (error, negative)
    80h-20h = 60h (error, positive)
    (-128-32 = 96)

    With saturation
    70h+20h = 7Fh (maximum value)
    80h-20h = 80h (minimum value)
- Used for:
  - Signal processing
  - Multimedia processing
- Typical amplification operation

  $U_e = U_i * A$



$$A = -\frac{R2}{R1}$$

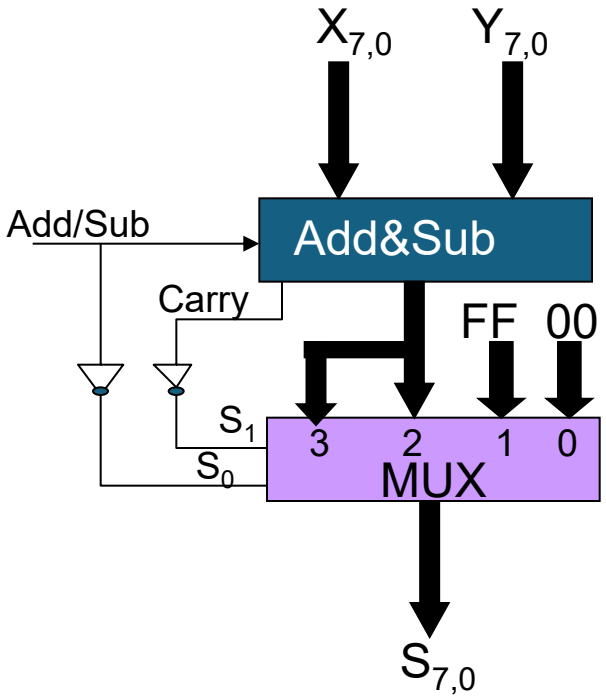
Supply: +10V; -10V, U_i=0.05 V; A=100 =>U_e = 5V

U_i=1.00 V; A=100 =>U_e = 10V !!! – positive saturation

Saturating Addition and Subtraction

- Saturating addition and subtraction for 8-bit unsigned representation
- the result is selected by a multiplexer:
  - Carry (C) = 0 => correct result
  - C = 1 and addition => overflow, result = FFh
  - C = 1 and subtraction => underflow, result = 00h

C	Add/ Sub	Operation	Result	S_1	S_0
0	0	adding	Correct X+Y	1	X
0	1	subtract	Correct X-Y	1	X
1	0	adding	Overflow FFh	0	1
1	1	subtract	Underflow 00h	0	0



C \ Add/Sub, S_0	0	1
0	X	X
1	1	0

C \ Add/Sub, S_1	0	1
0	1	1
1	0	0

Multiplication

- Multiplication = repeated addition

```
    1100 *  12 *
    1010    10
    0000
   1100
  0000
1100
1111000 = 78H = 120
```

Problems:

- requires a 2n-bit adder
- partial products placed at different positions

Modified multiply:

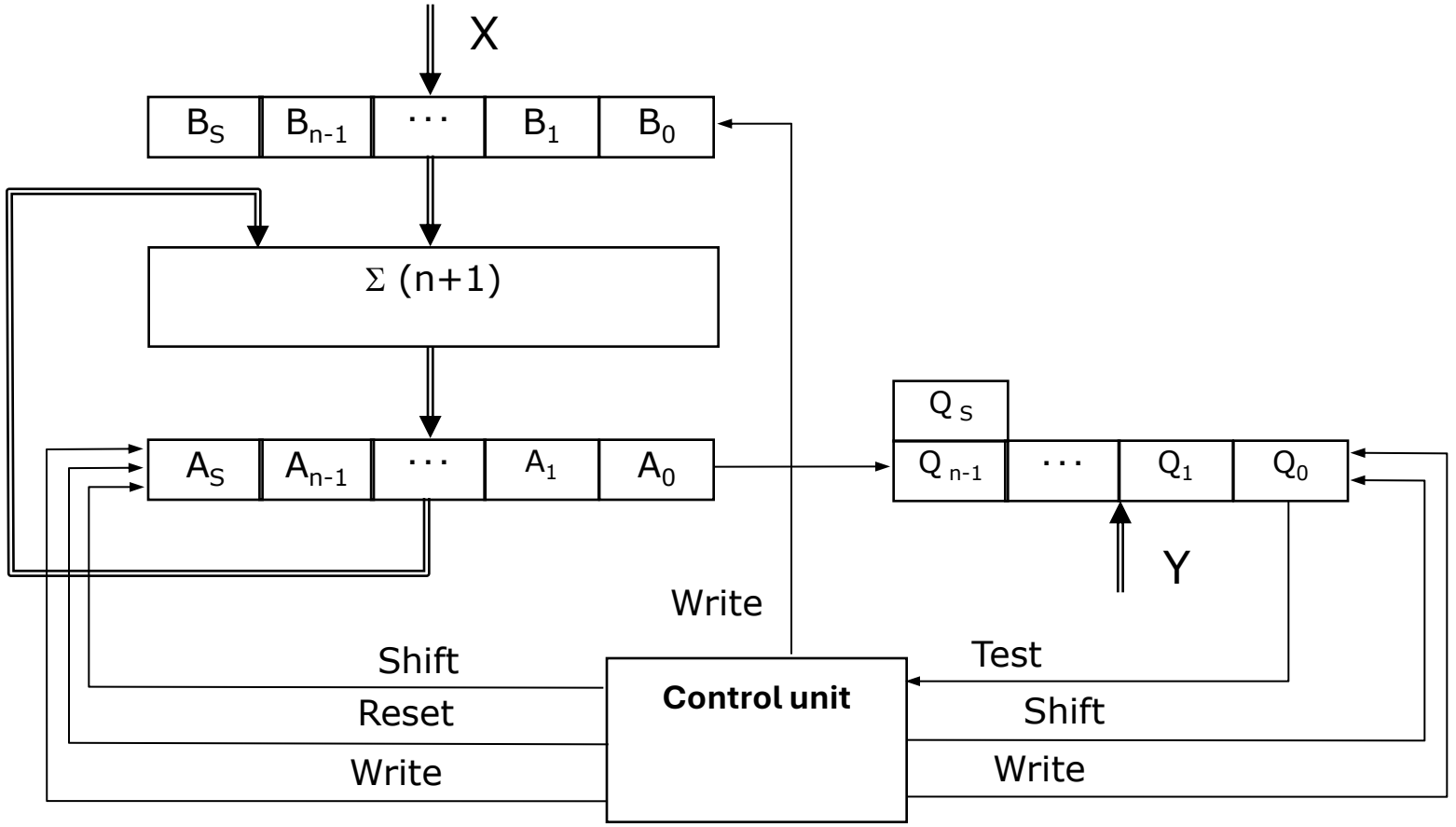
	00000000	Accumulator (AC)
"0" →	0000000 0	shift right
"1" →	1100	add
	0001100 0	partial result
	000110 00	shift right
"0" →	00011 000	shift right
"1" →	1100	add
	1111 000	final product

Solution: shift the partial result to the right and add each partial product in the same position

Advantages:

- only an n-bit adder is needed
- partial products in the same position

Multiplier


X
B_S B_{n-1} ⋯ B_1 B_0
Σ (n+1)
A_S A_{n-1} ⋯ A_1 A_0
Q_S
Q_{n-1} ⋯ Q_1 Q_0
Y
Write
Shift
Reset
Write
Control unit
Test
Shift
Write

Multiplication Algorithm

1. Write the operands into registers ($B \leftarrow X$, $Q \leftarrow Y$), clear the accumulator ($A \leftarrow 0$)
2. Complement negative numbers
3. Test Q_0
   - If $Q_0 = 0$, shift right A and Q
   - If $Q_0 = 1$, add A = B + A and shift right A and Q
4. Repeat step 3 until Y_{n-1} reaches Q_0. No shift is needed in the last step
5. $A_S = B_S \oplus Q_S$
6. If $A_S = 1$, complement the result

Multiplication with Booth's Algorithm

- Multiplies numbers in two's complement; the initial complementing is no longer needed
- For long sequences of 0s and 1s only shift operations are needed:
  - For 0s – evident from the previous scheme
  - For sequences of 1s:
    - Examples: 1111 = 10000 -1;
      11.1111 = 100.000 – 1
    - A sequence of 1s can be replaced by a sequence of zeros minus 1
- Only 0→1 or 1→0 transitions require an addition or subtraction, as follows:
  - If two consecutive bits of the second operand are:
    - 0 and 0 – shift right
    - 0 and 1 – add the operand and shift right
    - 1 and 0 – subtract the operand and shift right
    - 1 and 1 – shift right

3 × (−4), with m = 3 and r = −4, x = 4 and y = 4:

- m = 0011, -m = 1101, r = 1100
- A = 0011 0000 0
- S = 1101 0000 0
- P = 0000 1100 0
- Repeat the bit-pair check 4 times:
  1. P = 0000 110**0 0**.
     - P = 0000 0110 0. Arithmetic right shift.
  2. P = 0000 011**0 0**.
     - P = 0000 0011 0. Arithmetic right shift.
  3. P = 0000 001**1 0**.
     - P = 1101 0011 0. P = P + S.
     - P = 1110 1001 1. Arithmetic right shift.
  4. P = 1110 100**1 1**.
     - P = 1111 0100 1. Arithmetic right shift.
- The product is $1111\ 0100_2 = -12_{10}$.

Multiplication with a Look-up Table of Pre-computed Results

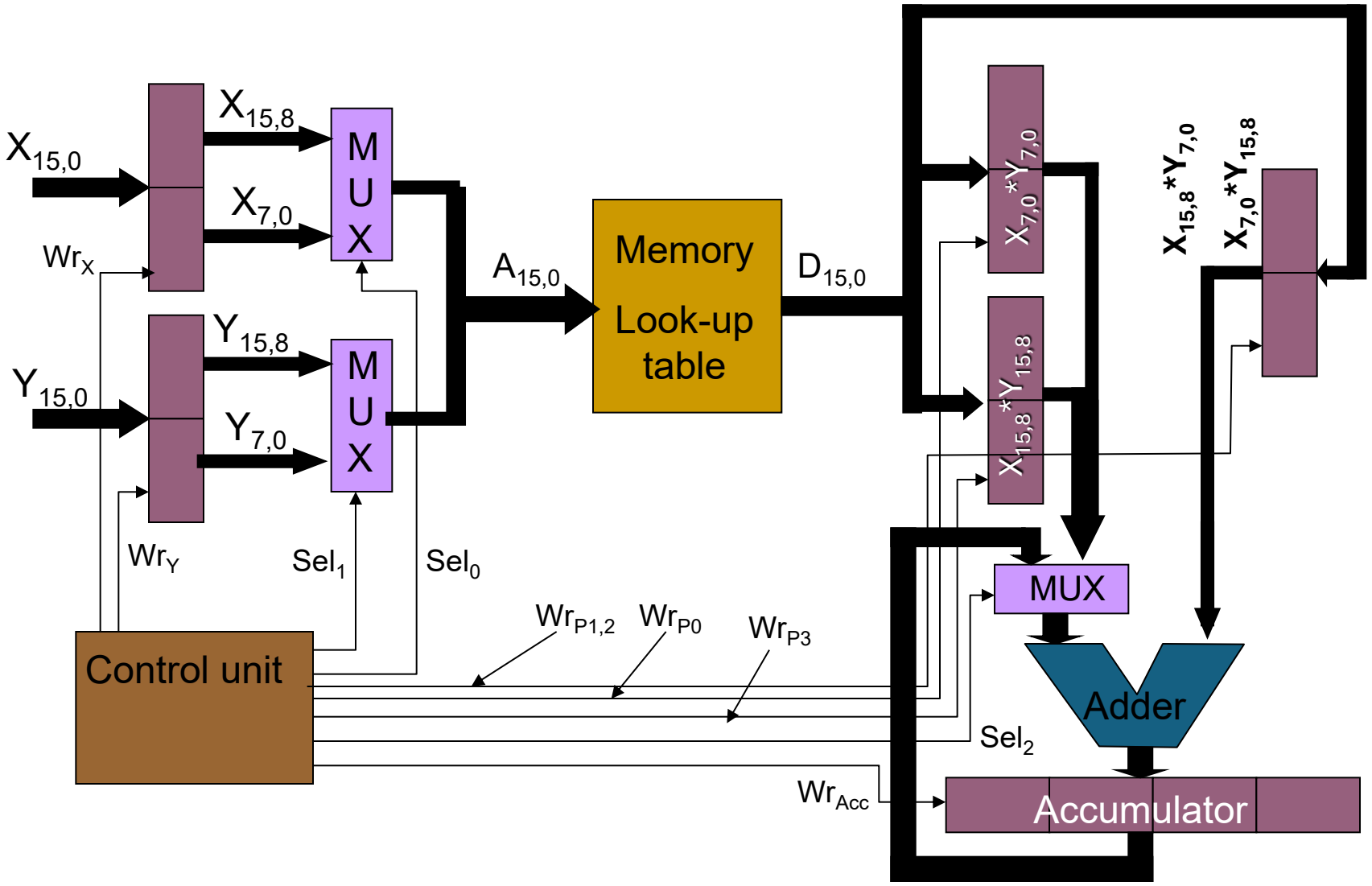
- Principle: all results are pre-computed and stored in a non-volatile memory
- Multiplication is a simple memory read
- The operands form the address of the location where the result is stored
- Problem: the memory size must be 2^{2n}
  - Examples:
    - 8*8 bits => 16 address lines => 2^{16} = 64KB
    - 16*16 bits => 32 address lines => 2^{32} = 4GB (too much)
  - Solution:
    - 8 × 8-bit multiplications in several steps to obtain 16-, 32- or 64-bit multiplications
    - Example:

$X = X_{15,8}\, X_{7,0} \quad Y = Y_{15,8}\, Y_{7,0}$

$P = X*Y = X_{7,0}*Y_{7,0} + X_{15,8}*Y_{7,0}*2^8 + X_{7,0}*Y_{15,8}*2^8 + X_{15,8}*Y_{15,8}*2^{16}$

Note: multiplications by 2^8 and 2^{16} are done by placing the result at the appropriate bit position; also, the first and last partial products can be combined in a single 32-bit register, with no addition needed

Look-up Table Multiplier Schematic

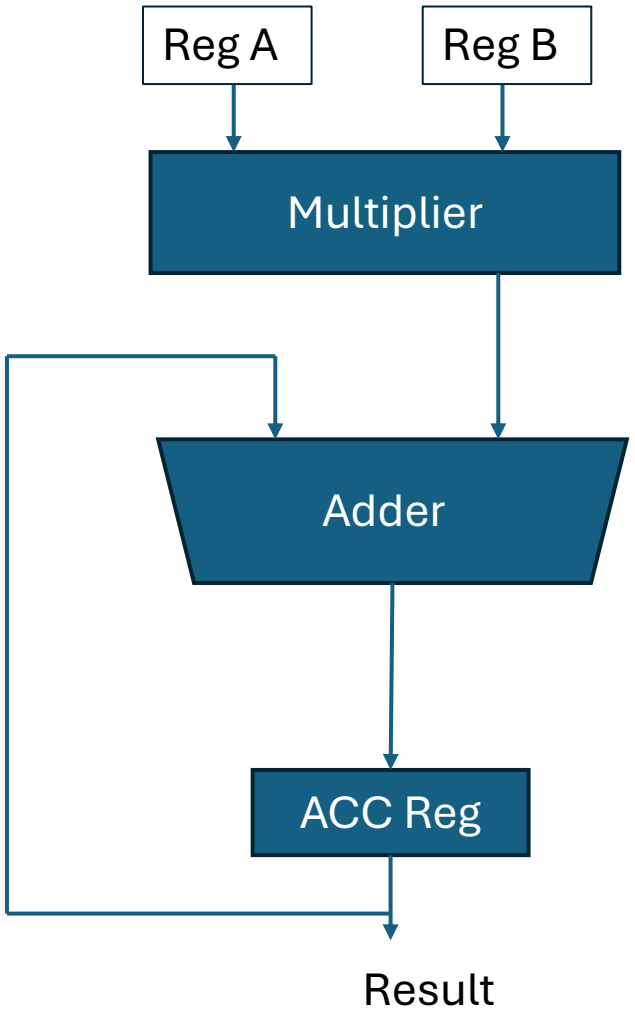


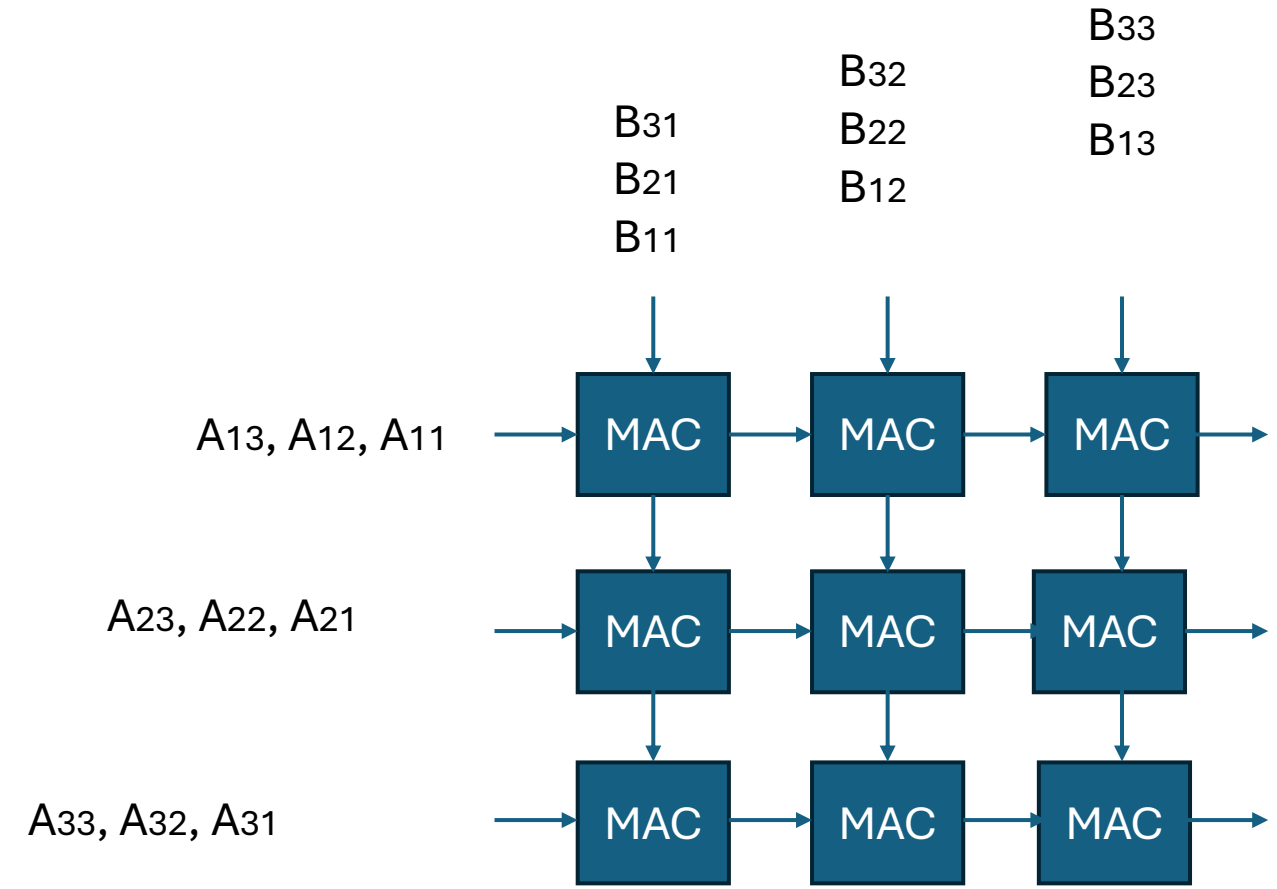
Look-up Table Multiplier Schematic

Step	*Wr_X*	*Wr_Y*	*Wr_{P0}*	*$Wr_{P1,2}$*	*Wr_{P3}*	*Wr_{Acc}*	*Sel_0*	*Sel_1*	*Sel_2*	*Description*
1	1	1	0	0	0	0	0	0	0	***Load operands***
2	0	0	1	0	0	0	0	0	0	***Write P0***
3	0	0	0	0	1	0	1	1	0	***Write P3***
4	0	0	0	1	0	0	1	0	0	***Write P1***
5	0	0	0	0	0	1	0	1	0	***Acc=P0+ P3 +P1***
6	0	0	0	1	0	0	0	1	0	***Write P2***
7	0	0	0	0	0	1	0	0	1	***Acc=Acc+P2***

- Look-up table multiplication needs only 7 steps instead of 16–20
- It can be optimized further

Multiply and Accumulate

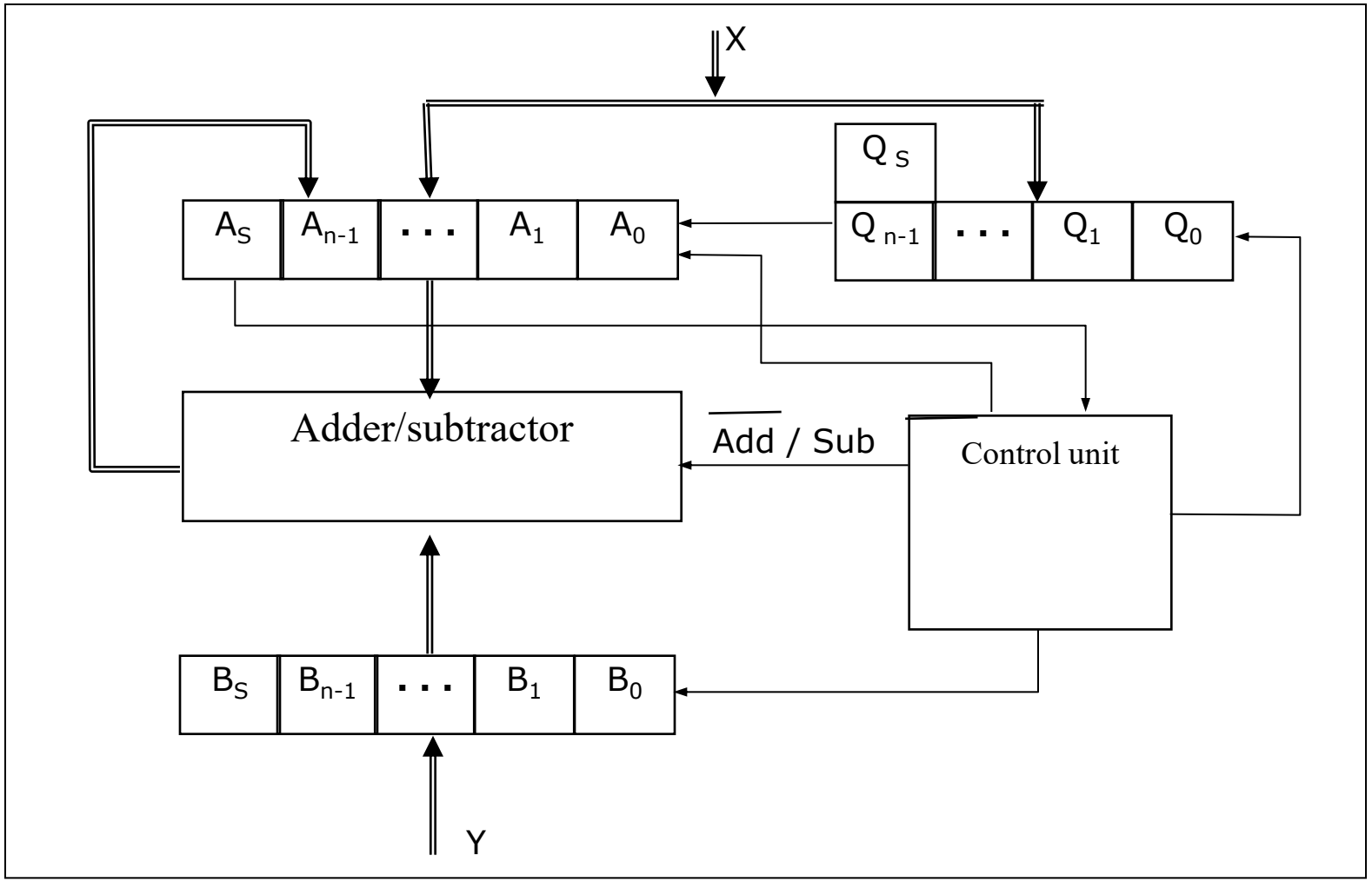




Division

- Several solutions:
  - Compare and subtract
    - Hard to implement at different positions
  - Subtract and restore the result (if needed)
    - Subtract the second operand from the most significant part of the first operand and
      - If the result is positive, it is fine (the quotient gets a 1),
      - Otherwise restore the result by adding back the second operand (the quotient gets a 0)
      - Disadvantage: some steps need 2 arithmetic operations (subtraction and addition)
  - Subtraction without restoring the partial remainder
    - Subtract B from the partial remainder: R’ = R - B
    - If a wrong subtraction was done in the previous step, the correction is made in the next step by adding the second operand instead of subtracting it
    - With correction: ((R-B) +B)*2 - B = R*2+B ; A shifted one position to the left
    - Without correction: (R – B)*2 + B = R*2 – B
    - Advantage: each step needs at most one subtraction or addition

Division Circuit for the 2nd Method – Restoring the Partial Remainder



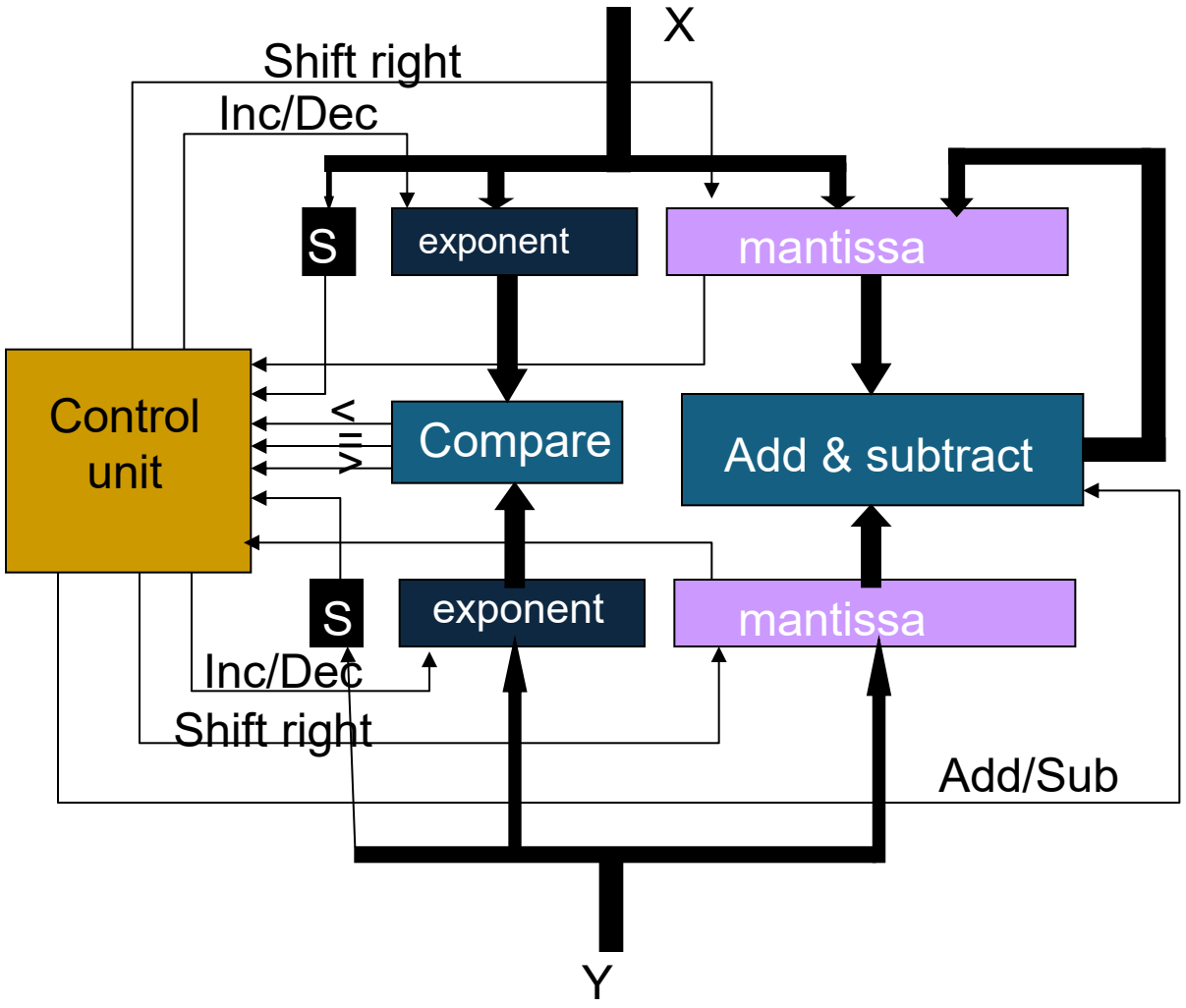
Restoring Division Algorithm

1. Load the first operand into A and Q; load the second operand into B
2. Write $A_S \oplus B_S$ in Q_S.
   If $A_S = 1$, complement A, Q
   If $B_S = 1$, complement B
3. Tests:
   A ≥ B, overflow
   B = 0, division by 0
   A = 0 and Q < B, result = 0
4. Shift A, Q left and put 0 in Q_0
5. Subtract B from A and put the result in A.
   if $A_S = 0$ (positive result) , shift A, Q left and put 1 in Q_0
   else $A_S = 1$ (negative result), add B to A, shift A, Q left and put 0 in Q_0
6. Repeat step 5 n times
7. Round the result. If A ≥ B, add 1 to the complement of Q
8. If $Q_S = 1$ complement Q

Floating-Point (FP) Arithmetic Operations

- Floating-point representation of a number:
  - For a subset of the real numbers
  - IT IS NOT THE REPRESENTATION OF THE REAL NUMBERS of mathematics !!!!!
  - 3 fields in the representation:
    - Sign – 1 bit
    - Exponent – the magnitude of the number
    - Mantissa – a few significant digits of the number
  - Many anomalies and precision problems:
    - Operating on numbers of very different magnitudes can produce rounding errors:
      - M+m-M = 0 ; M-M+m = m
    - Decimal fractions, in most cases, have no exact representation
      - Example: 0.3 has no exact floating-point representation

Floating-Point Adder/Subtractor



Floating-Point Addition

1. Load the operands
2. Compare the exponents (5 cases):
   - $e_x = e_y$, add the mantissas and copy the exponent
   - $e_x > e_y$ and $(e_x – e_y) <$ the number of mantissa bits, then mantissa m_y is aligned by shifting it e_x-e_y positions to the right;
   - $e_x >> e_y$ and $(e_x – e_y) \geq$ the number of mantissa bits, then X is copied to the result (Y is too small); go to step 4
   - $e_x < e_y$ and $(e_y – e_x) <$ the number of mantissa bits, then mantissa m_x is aligned by shifting it e_y-e_x positions to the right;
   - $e_x << e_y$ and $(e_y – e_x) \geq$ the number of mantissa bits, then Y is copied to the result (X is too small); go to step 4
3. Add the mantissas
4. Re-align (normalize) the result if needed: shift the resulting mantissa right or left until the integer part is 0 and the first bit after the binary point is 1; at the same time, increment or decrement the exponent according to the shift

Floating-Point Multiplication and Division

- Multiplication:
  - Add the exponents
  - Multiply the mantissas
  - Adjust the result (shift the mantissa left and decrement the exponent if needed)
- Division:
  - Subtract the exponents
  - Divide the mantissas
  - Adjust the result (if needed)

Arithmetic Pipeline

- Split the problem into sub-problems that can be solved in parallel

- Advantages: improves the execution time of operations
  - reduces the number of clock cycles needed to produce a new result
  - increases the throughput at the circuit's input and output

- More precisely:
  - a classic floating-point adder needs N clock cycles to produce a result – it can take a new set of operands only after the previous result has been computed (one new operand set every N clock cycles)
  - with pipelining, the adder can take a new set of operands even if the previous operation is not finished, once every C (C<N) clock cycles

Arithmetic Pipeline for Floating-Point Addition

